

M06 Learner Book

Git + GitHub + Linux/WSL Engineering Foundations | Release Candidate 1 | 13 controlled lessons

M06-L01 - Verify WSL2 and the Linux engineering workspace

Learning objective

Verify the Linux environment and choose a safe project location.

Why this matters in Data Engineering

Most later local engineering tools assume a Linux-compatible shell. The goal is a predictable workspace, not repeated reinstall attempts.

Understand

- WSL is a Linux environment integrated with Windows; it is not a second unrelated computer.
- Use `wsl --status` / `wsl -l -v` from PowerShell and `whoami/pwd/uname` from Linux to verify reality.
- Keep course repositories in the Linux filesystem by default; use `/mnt/c` intentionally for exchange with Windows.
- Record environment evidence before changing system configuration.

Try / inspect

```
PowerShell> wsl --status
PowerShell> wsl -l -v
Ubuntu$ whoami
Ubuntu$ pwd
Ubuntu$ uname -a
```

Common failure

Reinstalling or running admin commands because a tutorial says so without first verifying the existing environment.

Your turn - independent proof

Create `~/de-course/m06`, record distro/WSL version and current path, and explain where future repositories will live.

Move on when

You can open WSL, locate the workspace and state the Windows-vs-Linux path boundary without guessing.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L02 - Linux navigation and safe file operations

Learning objective

Navigate, create, copy, move and remove files with location awareness.

Why this matters in Data Engineering

A data engineer routinely works through terminals and automation where the current directory and target path determine what changes.

Understand

- pwd is a primary safety check before destructive file operations.
- Relative paths depend on the current directory; absolute paths do not.
- ls -la reveals hidden files such as .gitignore and .env.example.
- Use cp/mv/rm only after confirming the target; avoid unexplained recursive deletes.

Try / inspect

```
pwd
ls -la
mkdir sandbox
cd sandbox
touch note.txt
cp note.txt note-copy.txt
mv note-copy.txt moved.txt
rm moved.txt
```

Common failure

Running rm against the wrong directory or using a copied recursive command without understanding its expansion.

Your turn - independent proof

Build a nested sandbox, copy/move/delete only inside it, then explain each path you used.

Move on when

You can predict the exact target of a file command before running it.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L03 - Pipes, redirection and data inspection

Learning objective

Combine small shell tools to inspect data and capture command output.

Why this matters in Data Engineering

Fast inspection catches wrong paths, row-count surprises and format defects before heavier code runs.

Understand

- A pipe sends stdout from one process to another.
- > replaces a file; >> appends.
- head/tail/grep/wc/sort/uniq are useful for quick evidence, not substitutes for full parsing.
- Redirect logs/output deliberately and preserve stderr when diagnosing failures.

Try / inspect

```
head data/raw/orders.csv
wc -l data/raw/orders.csv
grep "000" data/raw/orders.csv
printf "a\nb\na\n" | sort | uniq -c
python3 src/pipeline.py > run_output.txt
```

Common failure

Treating shell line counts as business-row counts without accounting for headers or quoted multiline records.

Your turn - independent proof

Use a pipeline to count repeated values and redirect one program run to a file, then inspect the saved output.

Move on when

You can explain what flows through the pipe and where redirected output goes.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L04 - Permissions, packages, sudo and command safety

Learning objective

Use permissions/elevation intentionally and recognize high-risk commands.

Why this matters in Data Engineering

Local tooling often fails because of ownership or permissions. Random sudo/chmod fixes create harder security and reproducibility problems.

Understand

- sudo changes privilege context; ordinary project Python/Git work rarely needs it.
- Read permissions with ls -l before changing them.
- chmod 777 is not a universal fix; grant only required access.
- Package-manager commands change the environment and should be recorded.

Try / inspect

```
ls -l
id
git --version
python3 --version
# only when justified:
sudo apt update
```

Common failure

Using sudo for routine project commands and accidentally creating root-owned project files.

Your turn - independent proof

Inspect file permissions in the seed project, identify which commands need no elevation, and document one safe package update check.

Move on when

You can explain why a command needs elevation before using sudo.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L05 - Git repository state: init, status, add, commit and log

Learning objective

Understand working tree, staging area and committed history.

Why this matters in Data Engineering

Version control evidence is only trustworthy when you know what is staged, committed, ignored and still untracked.

Understand

- git status is the first diagnostic command.
- git add chooses content for the next snapshot; it does not save history by itself.
- A commit is an immutable history object referenced by branches.
- Small purposeful commits make review and recovery easier.

Try / inspect

```
git init
git status
git add README.md src/pipeline.py
git commit -m "Initialize engineering lab"
git log --oneline --decorate
```

Common failure

Running git add . and committing unknown generated files or secrets because status was not inspected.

Your turn - independent proof

Initialize a fresh copy of the seed, make two focused commits and show the state after each.

Move on when

You can identify working-tree, staged and committed changes from git status/diff.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L06 - Branches, merges and conflict resolution

Learning objective

Create parallel history and resolve a real text conflict correctly.

Why this matters in Data Engineering

Merge conflicts require human judgment. Avoiding them entirely is less useful than learning a safe evidence-driven resolution process.

Understand

- A branch is a movable reference to commits.
- Conflicts mark competing edits; they do not mean the repository is corrupt.
- Resolve by editing to the intended final content, removing markers, staging and committing.
- Verify both content correctness and a clean repository after the merge.

Try / inspect

```
git switch -c docs-update
# edit + commit
git switch main
# conflicting edit + commit
git merge docs-update
git status
```

Common failure

Deleting conflict markers without checking which business content should survive.

Your turn - independent proof

Create a deliberate conflict in docs/architecture.md, resolve it, and prove the final file plus log are correct.

Move on when

You can resolve a fresh conflict without copying a memorized sequence.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L07 - GitHub remotes: push, fetch, pull and tracking

Learning objective

Connect local history to a remote and distinguish download, integrate and upload operations.

Why this matters in Data Engineering

Remote confusion can overwrite work or create unnecessary merge noise. Each operation has a different direction and effect.

Understand

- origin is a conventional remote name, not a special server.
- fetch updates remote-tracking knowledge without changing the current branch.
- pull normally fetches then integrates.
- push publishes local commits and may establish upstream tracking.

Try / inspect

```
git remote -v
git push -u origin main
git fetch
git log --oneline --all --decorate
git pull
```

Common failure

Pulling with unresolved local changes or assuming fetch automatically changes the working tree.

Your turn - independent proof

Connect to an empty GitHub repository, push main, make one remote-only change, fetch it, inspect, then integrate intentionally.

Move on when

You can explain fetch vs pull vs push from the direction of data and resulting repository state.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L08 - Pull requests, review and collaboration workflow

Learning objective

Use a branch and pull request as a reviewable change boundary.

Why this matters in Data Engineering

Professional engineering changes should be inspectable, discussable and reversible before they reach the shared main branch.

Understand

- A PR proposes a branch diff for review; it is not just a merge button.
- Keep scope small enough that another person can understand the intent.
- Read the Files changed view before merging.
- After merge, synchronize local main before starting new work.

Try / inspect

```
git switch -c readme-improvement
# edit
git add README.md
git commit -m "Document rerun behavior"
git push -u origin readme-improvement
```

Common failure

Mixing unrelated changes into one PR or merging without reading the final diff.

Your turn - independent proof

Open a small PR, review the diff, merge it, then prove local main contains the merged commit.

Move on when

You can explain the change intent, review the diff and synchronize after merge.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L09 - Python virtual environments inside the repository

Learning objective

Create a disposable isolated Python environment without committing it.

Why this matters in Data Engineering

Dependencies differ between projects. A reproducible repository records dependency declarations, not a copied .venv directory.

Understand

- `python -m venv` creates an isolated environment.
- Activation changes command resolution for the current shell.
- `which python` and `python --version` verify the active interpreter.
- .venv belongs in .gitignore; dependency declarations belong in Git.

Try / inspect

```
python3 -m venv .venv
source .venv/bin/activate
which python
python --version
pip freeze > requirements-lock.txt
deactivate
```

Common failure

Committing .venv or assuming a copied environment is portable across machines.

Your turn - independent proof

Create/activate/deactivate .venv, install one harmless package if needed, record dependencies and prove .venv is untracked.

Move on when

You can rebuild the environment after deleting .venv using version-controlled dependency information.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L10 - .gitignore, secrets and repository hygiene

Learning objective

Separate source/config examples from secrets, logs and runtime outputs.

Why this matters in Data Engineering

A single leaked credential or giant runtime artifact can invalidate an otherwise good portfolio repository.

Understand

- .env.example documents names with fake placeholders; .env is local and may contain secrets.
- .gitignore prevents new tracking but does not erase already committed history.
- git ls-files shows what is actually tracked.
- Use fake values in learning evidence; rotate/revoke any accidentally exposed real secret.

Try / inspect

```
git status --ignored
git check-ignore -v .env
git ls-files
git grep -n -I -E "(API_KEY|TOKEN|PASSWORD)" -- . ".*!*.example"
```

Common failure

Assuming adding a filename to .gitignore removes a secret that was already committed.

Your turn - independent proof

Create a fake .env, prove it is ignored, inspect tracked files and document what you would do if a real token had been committed.

Move on when

You can prove secret/runtime files are not tracked and state the remediation boundary for an exposed credential.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L11 - Diff, restore, revert and safe history repair

Learning objective

Choose a recovery command based on what has or has not been committed/published.

Why this matters in Data Engineering

Engineers make mistakes. Safe recovery depends on understanding whether you are discarding local edits, creating a new corrective commit, or rewriting shared history.

Understand

- git diff shows unstaged changes; git diff --staged shows staged changes.
- git restore can discard selected local file changes; inspect first.
- git revert creates a new commit that undoes an earlier commit and is safer for shared history.
- Reset/rebase history rewriting requires extra care after publication.

Try / inspect

```
git diff
git diff --staged
git restore --source=HEAD -- path/to/file
git revert <commit>
```

Common failure

Using git reset --hard or force push as a first response to a mistake without preserving/understanding work.

Your turn - independent proof

Make three controlled mistakes in a sandbox: unstaged edit, staged edit and committed bad change. Recover each with the least destructive appropriate method.

Move on when

Given a repository mistake, you can choose restore/revert/history rewrite and explain why.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L12 - Shell scripts, exit codes and environment-driven commands

Learning objective

Turn repeated terminal steps into a small script with visible success/failure behavior.

Why this matters in Data Engineering

Automation depends on exit status, predictable working directories and explicit configuration, not on someone watching the terminal.

Understand

- Exit code 0 conventionally means success; non-zero signals failure.
- set -e is useful but not a substitute for thoughtful error handling.
- Use quoted variables and explicit project-root logic.
- Environment variables carry configuration; secrets should not be echoed.

Try / inspect

```
#!/usr/bin/env bash
set -euo pipefail
ROOT="$(cd "$(dirname "$0")" && pwd)"
cd "$ROOT"
python3 src/pipeline.py
echo "pipeline complete"
```

Common failure

Writing a script that only works from one current directory or continues after a failed command while still printing success.

Your turn - independent proof

Create run_pipeline.sh for the seed project, make one intentional failure, and prove the exit code prevents false success.

Move on when

You can run the script from a fresh shell and interpret its exit status.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M06-L13 - Repository handoff, reproducibility and evidence

Learning objective

Prepare a repository another learner can clone, understand and run safely.

Why this matters in Data Engineering

A portfolio repository is an operational handoff. Code without setup, assumptions, validation and rerun behavior is incomplete evidence.

Understand

- README should name purpose, setup, run command, inputs/outputs and validation.
- Record dependencies and required environment variables without secret values.
- Include a small safe dataset or reproducible generator.
- Provide evidence of clean status, tests/run output and known limitations.

Try / inspect

```
git status
python3 src/pipeline.py
git log --oneline -5
# README: setup -> run -> validate -> rerun -> limitations
```

Common failure

A repository that works only because of untracked local state or undocumented setup steps.

Your turn - independent proof

Hand your seed repository to a clean directory (or clone it) and reproduce the run using only version-controlled instructions.

Move on when

A fresh clone can be run and validated without relying on your memory or hidden files.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.